

UNIT I: ASYMPTOTIC ANALYSIS, ARRAYS & STACK / QUEUE LINEAR STRUCTURES

Weightage: 25% | Syllabus Section 1

1. Asymptotic Complexity & Master Theorem

Bounds: Big-O (Upper): $f(n) \leq c \cdot g(n)$; Big-Ω (Lower): $f(n) \geq c \cdot g(n)$; Big-Θ (Tight): $c_1g(n) \leq f(n) \leq c_2g(n)$.

Master Theorem: $T(n) = aT(n/b) + \Theta(n^k \log^p n)$, where $a \geq 1, b > 1$.

- If $\log_b a > k \implies T(n) = \Theta(n^{\log_b a})$
- If $\log_b a = k \implies T(n) = \Theta(n^k \log^{p+1} n)$
- If $\log_b a < k \implies T(n) = \Theta(n^k \log^p n)$

2. Array Memory Layout & Addressing

1D Array: $\text{Addr}(A[i]) = B + (i - \text{LB}) \cdot W$.

2D Row-Major: $\text{Addr}(A[i][j]) = B + [(i - L_r)C + (j - L_c)]W$.

2D Column-Major: $\text{Addr}(A[i][j]) = B + [(j - L_c)R + (i - L_r)]W$.

Sparse Matrix: Triplet form (*row, col, value*). CSR / CSC compressed formats save > 75% memory.

3. Stacks, Queues & Evaluation

Stack (LIFO): Push/Pop $O(1)$. Applications: Function call stack, Infix to Postfix conversion, Balanced Parentheses matching.

Queue (FIFO): Enqueue at Rear, Dequeue at Front $O(1)$. Circular Queue avoids memory leakage: $\text{rear} = (\text{rear} + 1) \pmod N$.

Priority Queue: Implemented via Binary Min/Max Heap: Insert $O(\log n)$, Extract-Min $O(\log n)$.

UNIT II: LINKED LISTS & NON-LINEAR TREES (BST, AVL & HEAPS)

Weightage: 25% | Syllabus Section 2

1. Singly, Doubly & Circular Linked Lists

Singly LL: Insert at Head $O(1)$, Search $O(n)$, Delete $O(n)$.

Doubly LL: Two pointers (*prev, next*). Allows bidirectional traversal; deletion of given node p is $O(1)$.

Circular LL: Last node points back to Head (*tail* → *next* = *head*). Excellent for Round-Robin schedulers.

2. Binary Search Trees (BST) & Traversals

BST Property: For every node: Left < Root < Right. Inorder traversal strictly yields sorted sequence!

Traversals: Inorder (L-Root-R), Preorder (Root-L-R), Postorder (L-R-Root). Time $O(n)$.

Search/Insert/Delete: Best/Avg $O(\log n)$; Worst-case $O(n)$ (skewed degenerative tree).

3. AVL Trees & Self-Balancing Rotations

Balance Factor (BF): $BF = \text{Height}(\text{Left}) - \text{Height}(\text{Right}) \in \{-1, 0, +1\}$. Strict height $h \leq 1.44 \log_2 n$.

Four Rotations:

- LL Imbalance ($BF = +2, +1$) → Single Right Rotation.
- RR Imbalance ($BF = -2, -1$) → Single Left Rotation.
- LR Imbalance ($BF = +2, -1$) → Left on child, Right on root.
- RL Imbalance ($BF = -2, +1$) → Right on child, Left on root.

UNIT III: GRAPH ALGORITHMS & SHORTEST PATHS

Weightage: 25% | Syllabus Section 3

1. Graph Representations & Traversals

Representations: Adjacency Matrix ($O(V^2)$ space), Adjacency List ($O(V + E)$ space, optimal for sparse graphs).

BFS (Breadth-First): Queue-based. Level-order expansion. Time $O(V + E)$. Finds shortest path in unweighted graphs.

DFS (Depth-First): Stack/Recursion-based. Time $O(V + E)$. Applications: Cycle detection, Topological sort in DAGs.

2. Minimum Spanning Trees (MST)

Kruskal's Algorithm: Greedy edge-selection. Sort edges, pick minimum weight avoiding cycles using Disjoint Set Union (DSU). Time: $O(E \log E) = O(E \log V)$.

Prim's Algorithm: Vertex-growing greedy strategy from initial root. Priority queue implementation: $O((V + E) \log V)$. Best for dense graphs.

3. Shortest Path Algorithms

Dijkstra: Single-source shortest path for non-negative weights. Greedy relaxation: $d[v] = \min(d[v], d[u] + w(u, v))$. Time: $O((V + E) \log V)$.

Bellman-Ford: Handles negative edge weights & detects negative cycles. Relax all edges $(V - 1)$ times. Time: $O(V \cdot E)$.

Floyd-Warshall: All-pairs dynamic programming: $D_{ij}^{(k)} = \min(D_{ij}^{(k-1)}, D_{ik}^{(k-1)} + D_{kj}^{(k-1)})$. Time: $O(V^3)$.

UNIT IV: SORTING, SEARCHING & HASHING TECHNIQUES

Weightage: 25% | Syllabus Section 4

1. Comparative Sorting Master Matrix

Algorithm	Best	Average	Worst	Space	Stable?
Bubble / Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

2. Hashing & Collision Resolution Strategies

Hash Functions: Division: $h(k) = k \pmod m$; Multiplication: $h(k) = \lfloor m(kA \pmod 1) \rfloor$.

Load Factor: $\alpha = n/m$ (n items, m slots). Expected search in chaining: $\Theta(1 + \alpha)$.

Collision Resolution:

- *Separate Chaining:* Linked list per bucket.
- *Open Addressing (Linear):* $h(k, i) = (h'(k) + i) \pmod m$.
- *Quadratic Probing:* $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \pmod m$.
- *Double Hashing:* $h(k, i) = (h_1(k) + i \cdot h_2(k)) \pmod m$.